1448. Count Good Nodes in Binary Tree

n | Category: Trees | LeetCode: https://leetcode.com/problems/count-good-node

Problem

- A node is "good" if no node on the path from the root to it has a greater value. Count good nodes.

Intuition

- Pass the maximum value seen on the path down the recursion. A node is good when node.val >= maxSoFar.

Brute Force

- For each node, walk the path from the root again: O(n h).

Optimal Approach

- Dfs(node, max): if null return 0.
- Good = node.val >= max ? 1 : 0; newMax = max(max, node.val).
- Return good + dfs(left, newMax) + dfs(right, newMax).

Worked Example

```
[3,1,4,3,null,1,5].

3 (max -inf) good   max=3
 1 (3) no             ; 3 under 1 (max 3) good
 4 (3) good  max=4    ; 1 (4) no ; 5 (4) good
total = 4
```

Java Solution

```java
public class CountGoodNodes {
    public int goodNodes(TreeNode root) { return dfs(root, Integer.MIN_VALUE); }

    private int dfs(TreeNode n, int max) {
        if (n == null) return 0;
        int good = n.val >= max ? 1 : 0;
        max = Math.max(max, n.val);
        return good + dfs(n.left, max) + dfs(n.right, max);
    }
}
```

Complexity

- Time: O(n). Space: O(h).

Pitfalls

- Equal values count as good (>=).